SF2524
Matrix computations for large-scale systems
Homework 2

Group 34
Ville Wassberg
Klara Zimmerman

November 2024

Problem 1

In this exercise we implement GMRES based on the Arnoldi method. This is done by computing $z*$ such that $z* = argmin_{z \in R^m} ||\underline{H}_m z - e_1||b||||_2$, and computing the approximate solution $\tilde{x} = Q_m z*$ in each iteration of Arnoldi. We then look at the relative residual norm, $||Ax_m - b||/||b||$ and the relative error $||x_* - x_m||/||x_m||$ (where $x*$ is the true solution) in each iteration.

We use GMRES to find an approximation of the vector x in the system of equations modelled by $A_\alpha x = b$. $A_\alpha \in R^{100 \times 100}$ is a sparse matrix, and we shift the diagonal elements of A_α with different values of α. Figure 1 shows how the convergence of GMRES is affected by these shifts.

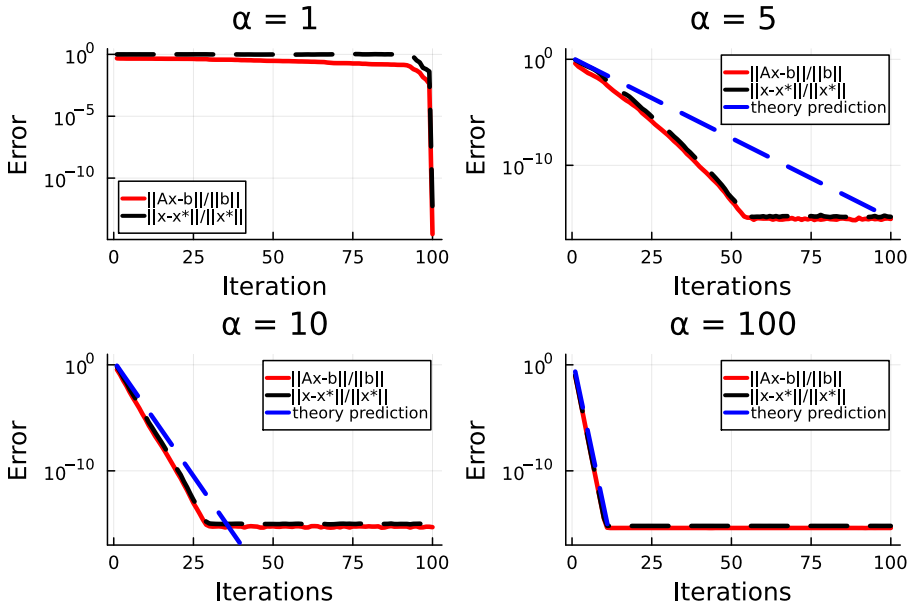

Figure 1: Errors from iterates of GMRES to solve a problem $A_\alpha x = b$, where the diagonal elements of A_α are shifted with different values α, along with the theoretical convergence bounds.

In order to find a bound for the convergence factor, we plot the eigenvalues of each matrix A_α. By the theory of the "single oulier" convergence of GMRES (see exercise 3b below), where λ_1 is the outlier and $\lambda_2, ..., \lambda_n$ are the remaining eigenvalues of A_α, we have the upper bound

$$B(A_\alpha) = \frac{|\lambda_1^{(\alpha)} - c^{(\alpha)}| + r^{(\alpha)}}{|\lambda_1^{(\alpha)}|} \left(\frac{r^{(\alpha)}}{|c^{(\alpha)}|} \right)^{m-1}.$$

From the plots in figure 2, we get the following bounds:

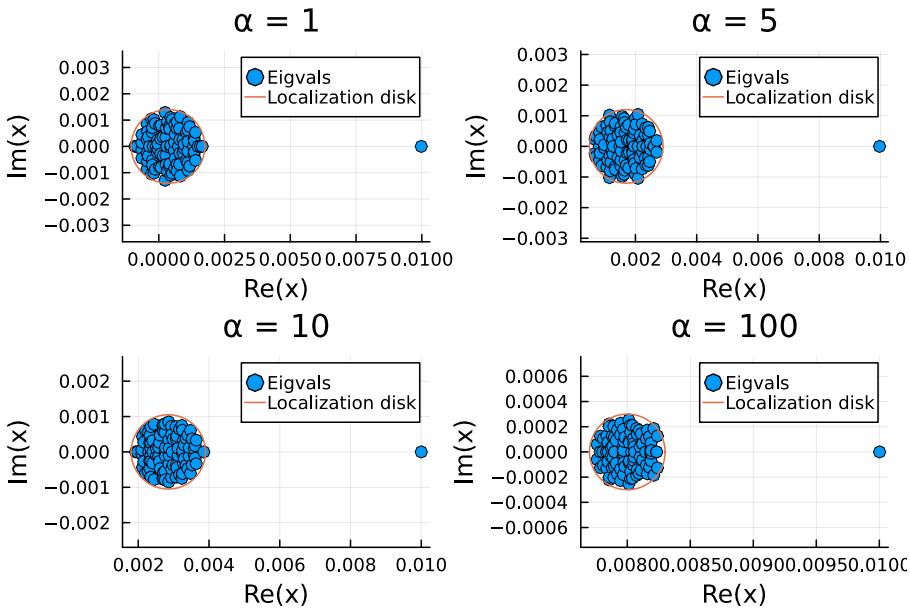

Figure 2: Eigenvalues of the matrices A_α.

- $B(A_{\alpha=1}) \approx \frac{|0.01-0.00035|+0.0014}{|0.01|}\left(\frac{0.0014}{0.00035}\right)^{m-1} = 1.115 \times 4.0^{m-1}$
- $B(A_{\alpha=5}) \approx \frac{|0.01-0.0017|+0.0012}{|0.01|}\left(\frac{0.0012}{0.0017}\right)^{m-1} = 0.95 \times (0.706)^{m-1}$
- $B(A_{\alpha=10}) \approx \frac{|0.01-0.00285|+0.00105}{|0.01|}\left(\frac{0.00105}{0.00285}\right)^{m-1} = 0.82 \times (0.368)^{m-1}$
- $B(A_{\alpha=100}) \approx \frac{|0.01-0.008|+0.0003}{|0.01|}\left(\frac{0.0003}{0.008}\right)^{m-1} = 0.229 \times (0.0375)^{m-1}$

This tells us that for $\alpha \in \{5, 10, 100\}$, GMRES will converge, since the convergence factor is less than 1. These bounds are plotted along with the residual and error in each plot in figure 1. However, when $\alpha = 1$, the convergence factor is greater than 1, which implies that we cannot guarantee convergence. As we can see in figure 1, GMRES for the matrix $A_{\alpha=1}$ does in fact not converge until the number of iterations equals the dimension of $A_{\alpha=1}$, which in this example is 100.

GMRES

		$n = 200$		$n = 500$		$n = 1000$	
	m	resnorm	time	resnorm	time	resnorm	time
$\alpha = 1$	5	3.790	0.0001	6.487	0.0005	9.119	0.0032
	10	3.717	0.0002	6.426	0.0011	9.094	0.0063
	20	3.534	0.0005	6.317	0.0026	9.035	0.0134
	50	3.152	0.0027	6.163	0.0091	8.931	0.0392
	100	2.543	0.0086	5.861	0.0277	8.674	0.0925
$\alpha = 100$	5	5.101e-6	0.0001	9.206e-5	0.0005	6.816e-4	0.0031
	10	1.094e-12	0.0002	1.917e-10	0.0011	7.350e-9	0.0063
	20	5.826e-15	0.0005	1.089e-14	0.0026	2.153e-14	0.0130
	50	5.985e-15	0.0027	1.093e-14	0.0090	2.196e-14	0.0372
	100	5.124e-15	0.0086	1.096e-14	0.0272	2.124e-14	0.0860

Table 1: GMRES Results for $A_{\alpha=1}$ and $A_{\alpha=100}$, where resnorm $= ||Ax - b||_2$, time is the CPU and m is the number of iterations.

Backslash

	$n = 200$		$n = 500$		$n = 1000$	
	resnorm	time	resnorm	time	resnorm	time
$\alpha = 1$	2.446e-11	0.0028	7.730e-10	0.0217	1.486e-9	0.1012
$\alpha = 100$	5.283e-15	0.0026	1.196e-14	0.0215	2.332e-14	0.0870

Table 2: Backslash Results for $A_{\alpha=1}$ and $A_{\alpha=100}$, where resnorm $= ||Ax - b||_2$, time is the CPU and m is the number of iterations.

We now compare GMRES to the backslash command in Julia, for matrices of different dimensions and $\alpha-$shifts. As we can see in Table 1 and Table 2, for $\alpha = 1$ we get satisfactory results using the backslash command, while GMRES does not provide very accurate results.

However, for $\alpha = 100$ we see convergence using GMRES. After 20 iterations, the results are of the same accuracy as when using the backslash command. Looking at Table 1 and Table 2, it is even preferable to use GMRES when solution accuracy is acceptable around 10^{-5} for $\alpha = 100$. Even for $n = 1000$, we only need $m = 10$ to reach this level of accuracy, which takes 0.0063 seconds. The same system takes more than 10 times as long to solve using the backslash command.

Problem 2

In this problem, we compare the convergence of the residual norm using three different methods: GMRES, Conjugate Gradient (CG), and CG with least squares (LSQ).

CG with LSQ is obtained through solving the minimization problem

$$z = \arg\min_{z \in \mathbb{R}^p} \|AXz - b\|,$$

where X is the matrix consisting of iterations from CG. Using this z, we form the approximation $x = Xz$. As can be seen in figure 3, all three methods seem to produce very similar residuals in each iteration, up until iteration 11, where the residual from CG with LSQ flattens out. This is due to the matrix $(A\,X_p)^T\,A\,X_p$ becoming nearly singular from iteration 11, forcing a regularisation constant to be added, which in this case was chosen to be $10^{-13}\cdot\mathbf{I}$ making the system to solve in each step after iteration 10 equal $(A\,X_p)^T\,A\,X_p + 10^{-13}\cdot\mathbf{I} = (A\,X_p)^T\,b$.

As can be seen in figure 4, CG produced slightly different results compared to the other two methods. This can however only be spotted when zooming in quite a bit. GMRES and CG with LSQ however seem to converge with the exact same norms (up to 15 decimals) up until the regularization term is added to CG with LSQ.

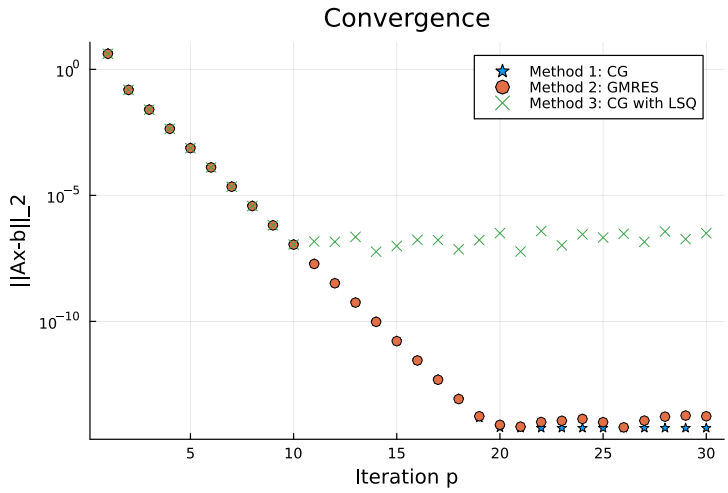

Figure 3: Residual norms of CG, GMRES and CG with LSQ.

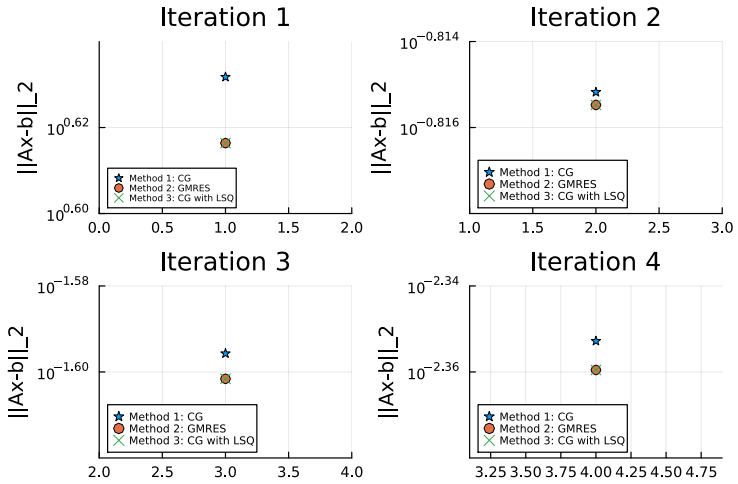

Figure 4: Residual norms of CG, GMRES and CG with LSQ - zoomed in on the first four iterations

According to the theory presented about convergence of CG, this method will converge proportionally to the square root of the condition number of the matrix A. That is, the number of iterations needed to achieve a certain level of accuracy for CG is proportional to

$$\sqrt{\kappa(A)} = 1.41. \tag{1}$$

As for the convergence of GMRES, it is proportional to the condition number of the matrix V, when A is diagonalizable and invertible and can be written $A = V\Lambda V^{-1}$. We indeed have a diagonalizable and invertible matrix A (since the eigenvectors of A are linearly independent and A is positive definite), thus we get that the convergence of GMRES is proportional to

$$\kappa(V) = 1.00. \tag{2}$$

Equations (1) and (2) show that the convergence of both CG and GMRES are very similar, and implies that they will both converge quickly. The slight difference spotted in figure 4 can be explained by the slightly larger bound for CG compared to GMRES.

When the approximation $x = Xz$ (where X contains the iterations from CG) is made in order to use CG with LSQ, we derive the matrix AX for solving the normal equations.

The same outputs of GMRES and CGLSQ can possibly be explained by that the outputs x_m are minimizers of the residual found in the Krylov subspace of

A and b, which also is where GMRES performs the minimization. The CGLSQ then performs the minimization in essentially the same subspace, since X_p is linear mapped by A.

In conclusion, CG with LSQ performs just as well as GMRES (up until the regularization term is needed). Depending on the accuracy tolerance in question, and assuming the number of iterations is our only measurement of how "fast" a method converges, both GMRES and CG with LSQ are satisfactory methods for solving this problem. However, as we will see in problem 5, CG is a much faster method when considering CPU time, thus this may even be considered a better method than GMRES.

Problem 3

(a) **201 CPU-socket application and iterative vs direct methods**

When solving boundary value problems, for example the steady-state heat equation for a CPU-socket, the resulting linear system is often large and sparse due to fine discretization. Direct methods, such as Gaussian Elimination, provide a solution with fixed complexity, but require the entire computation to complete before yielding a result. Alternatively, we can use iterative methods to gradually improve the solution, where each iteration is cheap to compute. Iterative methods are particularly effective for large, sparse systems like those encountered in the CPU-socket application.

(b) **215 GMRES convergence single outlier**

If a matrix A has eigenvalues $\lambda_1, ..., \lambda_n$, where all eigenvalues but λ_1 lie in a disc with center c and radius r, we call λ_1 an outlier. In this video, it is proved that finding these eigenvalues using the GMRES algorithm has the same rate of convergence as the corresponding problem without an outlier, that is $\left(\frac{r}{|c|}\right)^{m-1}$. In the case with the outlier, we multiply this convergence rate by the constant $\frac{|\lambda_1 - c| + r}{|\lambda_1}$, resulting in the upper bound

$$\min_{p \in P_m^0} \max_{j=1,..,n} |p(\lambda_j)| \leq \frac{|\lambda_1 - c| + r}{|\lambda_1} \left(\frac{r}{|c|}\right)^{m-1}.$$

(c) **216 Preconditioning for GMRES (part 1)**

For some systems $Ax = b$, GMRES converges very slowly or not at all. To get around this problem, we can use left preconditioning, where we find a matrix M such that $M^{-1}Ax = M^{-1}b$ converges. This is done without explicitly computing $B = M^{-1}A$. A good preconditioner M is a balance between $M \approx I$ which makes $M^{-1}z$ cheap to compute but has slow convergence, and $M \approx A$ which makes $M^{-1}z$ expensive to compute but with fast convergence. Common choices are the diagonal of A, an incomplete LU-factorization or a multigrid.

(d) **234 Preconditioning CG part 1**

In the Conjugate Gradient method, the matrix A must be symmetric and positive definite (spd). This means that we cannot use left conditioning or right conditioning to improve the convergence of the method, as this would result in non-spd matrices. However, we can combine the two methods, conditioning both from the left and right with a matrix R, so that $B = R^{-T}AR^{-1}$. This will preserve symmetry and positive definiteness. If R is selected such that $A \approx R^T R$, we get fast convergence of our problem.

(e) **235 Preconditioning CG part 2**

If we let $B = R^{-T}AR^{-1}$ and $c = R^{-T}b$, our original system $Ax = b$ can now be written as $B\hat{x} = c$. Inserting this into our original CG algorithm, we see that

new updates of the parameters can easily be found: $\hat{x}_m = R^{-1}x$, $\hat{p}_m = R^{-1}p_m$ and $\hat{r}_m = R^T r_m$. The updates $\hat{x}_{m+1}$, $\hat{p}_{m+1}$ and $\hat{r}_{m+1}$ can then then be modified accordningly, and the preconditioned CG (PCG) is found.

(f) **250 BiCG method derivation**

We can derive the Bilanczos Conjugate Gradient (BiCG) method from PCG by considering $Ax = b$ and an adjoint system $A^T\hat{x} = \hat{b}$. This gives us the system

$$\begin{pmatrix} 0 & A^T \\ A & 0 \end{pmatrix} \begin{pmatrix} x \\ \hat{x} \end{pmatrix} = \begin{pmatrix} \hat{b} \\ b \end{pmatrix},$$

with the preconditioner $M = \begin{pmatrix} 0 & I \\ I & 0 \end{pmatrix}$.

We can now partition the vectors in the PCG as follows:

$$r_m \leftarrow \begin{pmatrix} r_m \\ \hat{r}_m \end{pmatrix}, p_m \leftarrow \begin{pmatrix} p_m \\ \hat{p}_m \end{pmatrix}, z_m \leftarrow \begin{pmatrix} z_m \\ \hat{z}_m \end{pmatrix}, x_m \leftarrow \begin{pmatrix} x_m \\ \hat{x}_m \end{pmatrix},$$

and modify and add updates in the algorithm accordingly. This typically results in a method with erradic convergence, but a smaller conditioning number than that of CGN.

(g) **240CGNE - derivation of conjugate gradients normal equations**

The conjugate normal equations are derived by multiplying the non-symmetric problem $Ax = b$ with A^T from the left providing the new system

$$A^T A\, x = A^T b \Leftrightarrow B\, x = c,$$

where B is SPD, which can be seen by $z^T Bz = z^T A^T Az = (Az)^T Az = \|A\, z\|_2^2 > 0$. Applying the conjugate gradient to this system instead, without explicitly doing the matrix computations by keeping to the matrix vector products as $A^T\,(A\, x)$. When finding the minimiser of this system we get that it is also a minimiser of $\|Ax - b\|_2^2$ as GMRES, since

$$\|Bx{-}c\|_{B^{-1}}^2 = \|x{-}x_*\|_B^2 = (x{-}x_*)^T\, B\, (x{-}x_*) = (x{-}x_*)^T\, A^T\, A\, (x{-}x_*) = \|Ax{-}b\|_2^2.$$

Despite the similarities to GMRES, the minimisation is made over a different Krylov subspace which is generated by B and c instead of A and b, hence the behaviour of the algorithms are in general different.

Problem 4

To prove a step bound for CG that reduces error by a factor of 10^7, for the matrix $A \in \mathbf{R}^{n\times n}$, real and symmetric, with eigenvalues $\lambda_1 = 10$, $\lambda_2 = 10.5$ and $\lambda_i \in [2, 3]$, $i = 3, \ldots, 102$, we can start by notice that all eigenvalues are real and strictly positive, hence A is a symmetric positive definite (SPD) matrix. Therefore, we can use the condition number bound of CG, found in Block 2, that gives an error bound:

$$\|Ax_m - b\|_{A^{-1}} \leq 2\|b\|_{A^{-1}} \left(\frac{\sqrt{k}-1}{\sqrt{k}+1}\right)^m, \text{ where } k := \frac{\lambda_{max}}{\lambda_{min}}.$$

Hence, to improve the error by a factor of 10^7 we need to find m such that

$$10^7 \leq \frac{\|Ax_0 - b\|_{A^{-1}}}{\|Ax_m - b\|_{A^{-1}}} \leq \frac{2\|b\|_{A^{-1}} \left(\frac{\sqrt{k}-1}{\sqrt{k}+1}\right)^0}{2\|b\|_{A^{-1}} \left(\frac{\sqrt{k}-1}{\sqrt{k}+1}\right)^m} = \left(\frac{\sqrt{k}+1}{\sqrt{k}-1}\right)^m.$$

Thus, we have that

$$m \geq \frac{7}{\log\left(\frac{\sqrt{k}+1}{\sqrt{k}-1}\right)} \geq \frac{7}{\log\left(\frac{\sqrt{10.5/2}+1}{\sqrt{10.5/2}-1}\right)} \approx 17.2268.$$

This shows that 18 steps is enough to decrease the error by a factor of 10^7, assuming no roundoff errors nor premature breakdown.

Problem 5

We now want to compare GMRES to the Conjugate Gradient method with Normal Equations (CGN). We test these two methods on a system of equations $Ax = b$ where $A \in R^{n \times n}$ and $b \in R^n$ are provided by the file "cgn_illustration.mat", and $n = 10^5$. Because of these large dimensions, we do not have access to any "true" solution in this problem.

We begin by running both methods and saving the residual norm in each iteration. As seen in figure 5, GMRES converges after about 50 iterations, while CGN converges after about 65 iterations.

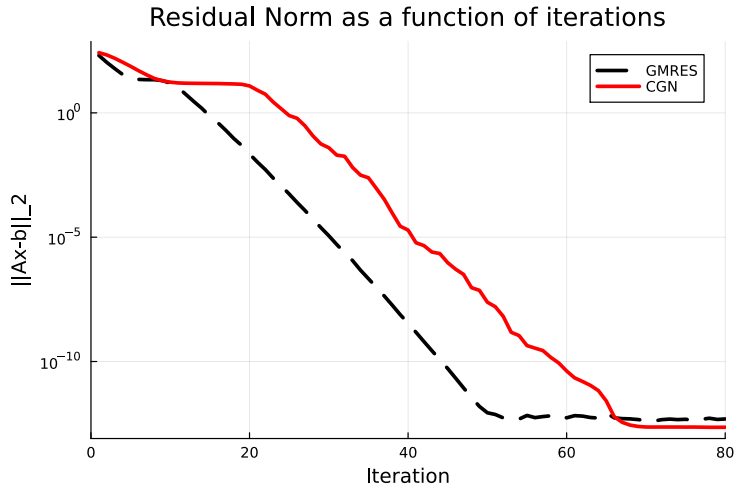

Figure 5: Relative residual norm of of GMRES and CGN as a function of iterations

Next, we look at the time it takes to reach convergence for the respective methods. This is done in Julia with the Int(time_ns()) command, after which the time is converted to seconds. When timing the methods, we make sure not to calculate the residual in each iteration, as the matrix-vector product needed for this is computationally expensive and not needed. As we can see in figure 6, convergence is reached by GMRES after more than 2 seconds, but by CGN after less than 0.5 seconds.

Theoretically, the convergence of CGN depends on the condition number of $A^T A$. Specifically, as a rule of thumb, the number of iterations needed for CGN to convergence to a certain accuracy is proportional to $\sqrt{\kappa(A^T A)} = \kappa(A)$. If this condition number is too large, CGN is unlikely to be a usefull method. Thus, we can assume from our results that the condition number of A is at least moderately small in our problem.

For GMRES, if A is diagonalizable and invertible, the convergence is influenced

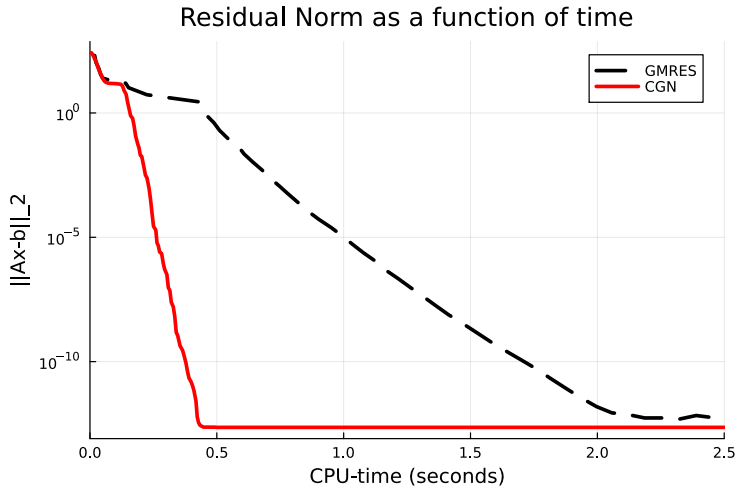

Figure 6: Relative residual norm of of GMRES and CGN as a function of CPU time

by the condition number of the eigenvector matrix V, where $A = V\Lambda V^{-1}$. Specifically, the rate of convergence depends on how well-conditioned V is. We thus infer from the observed relatively fast convergence of GMRES that $\kappa(V)$ is also relatively small.

When it comes to CPU timing, CGN is quite effective as it only requires two matrix-vector products per iteration. Assuming that the condition number of A is not too large, this method is hence very fast. Meanwhile, GMRES requires an orthogonalization (double Gram-Schmitt in our case) in each iteration, which significantly slows it down. As we see when comparing figures 5 and 6, although CGN takes a few more iterations to converge, the CPU time needed for convergence is less than than of GMRES.